Oblivious ML-Ops (OMLO): Privacy-Preserving Machine Learning Infrastructure via Oblivious RAM

Progress Report

Caden Wesley Roberts
University of California, Santa Cruz
cawrober@ucsc.edu
https://github.com/cadenroberts/OMLO

March 2026

Abstract

Machine learning training pipelines expose data-dependent memory access patterns that leak private information even when data values are encrypted. This report presents Oblivious ML-Ops (OMLO), a framework that integrates Path ORAM into PyTorch training workflows to hide these access patterns. We describe the threat model that motivates oblivious access during training, characterize the specific information an adversary can observe, quantify the overhead ORAM introduces, and provide a detailed comparison between plaintext and ORAM-backed training on CIFAR-10 with ResNet-18. Results show that ORAM preserves model accuracy ($\sim$93%) with no convergence degradation while introducing 90–180$\times$ per-epoch wall-time overhead dominated by encrypted block I/O.

1 Introduction

Privacy-preserving machine learning has received significant attention through techniques such as federated learning and differential privacy. These approaches protect *data values* during training: federated learning keeps raw data on-device, and differential privacy bounds what can be inferred about individual records from model outputs. However, neither technique addresses *metadata leakage*: the patterns of memory accesses, disk reads, and network requests that a training pipeline generates reveal information about the underlying data, independent of whether the data itself is encrypted.

OMLO addresses this gap by integrating Oblivious RAM (ORAM) into the data-loading layer of a standard PyTorch training pipeline. ORAM is a cryptographic primitive that provably hides which data items are accessed, how often, and in what order. This report focuses on three questions raised during review:

(1) Why is obliviousness necessary during training?

(2) What information can an adversary observe during training that has privacy requirements?

(3) What overhead does ORAM introduce, and how does ORAM-backed training compare to plaintext training?

The measured overheads (90–180$\times$ per epoch) are comparable to early secure-computation systems and establish a baseline for evaluating future optimizations such as concurrent ORAM and GPU-accelerated cryptography.

2 Related Work and Positioning

ORAM in secure ML training. ORAM has been applied to secure computation and inference [1, 2], but its integration into ML *training* pipelines remains underexplored. Prior work on privacy-preserving ML (federated learning, differential privacy, secure aggregation) protects data values or gradient statistics, not storage-level access patterns. OMLO integrates Path ORAM into the PyTorch data-loading layer to provide oblivious sample access during training. While ORAM has been widely studied in secure storage and computation systems, its direct integration into a modern ML training pipeline has received limited empirical evaluation.

ORAM in SGX and enclave training. Secure enclaves (e.g., Intel SGX) protect memory confidentiality and integrity from the OS and hypervisor, but they do *not* hide access patterns: an adversary observing the enclave's storage I/O still sees which blocks are read and in what order. Systems such as Gramine-PyTorch and Plinius run training inside enclaves for memory protection; OMLO addresses the complementary threat of storage-level access-pattern leakage. ORAM and enclaves are orthogonal: enclaves protect *what* is in memory; ORAM hides *which* blocks are accessed.

ORAM data loaders in prior systems. OBLIVIATOR [4] provides oblivious parallel operators for database joins in shared memory, not for ML data loading. Treebeard and similar ORAM datastores offer scalable oblivious storage but are not integrated with ML frameworks. PyORAM [6] provides the underlying Path ORAM implementation; OMLO contributes the `ORAMDataset` abstraction, serialization for variable-size samples, and the integration into PyTorch's training loop.

3 Why Obliviousness Is Needed During Training

3.1 Access Patterns as a Side Channel

A machine learning training loop repeatedly reads samples from a dataset. In standard PyTorch, the `DataLoader` fetches samples by index: each call to `__getitem__(i)` issues a memory or disk read at an address determined by sample index i. The *sequence* of indices accessed during training, and the *frequency* with which each index appears, constitutes an access pattern.

Even if every sample is stored encrypted at rest, the access pattern itself is observable to any entity that can monitor the storage layer: the operating system, the cloud hypervisor, a co-located tenant on shared hardware, or a compromised storage controller. This is the classic side-channel problem that ORAM was designed to solve [1, 2].

3.2 What Access Patterns Reveal

During training, access patterns leak information along several axes:

(a) Sample selection order. The DataLoader shuffles indices each epoch to produce a random permutation. An adversary observing the permutation learns which samples appear in the same mini-batch. Under uniform random shuffling this co-occurrence is unlikely to reveal class structure directly; however, pipelines that use stratified sampling, class-balanced batching, or curriculum-based ordering produce non-uniform permutations where batch composition correlates with private label or demographic attributes.

(b) Per-sample access frequency. Curriculum learning, hard-example mining, and data augmentation strategies cause certain samples to be accessed more frequently than others. An

adversary counting per-index access frequency can infer which samples the model finds "difficult," leaking information about data distribution and label noise.

(c) Data-dependent branching. Some training pipelines conditionally skip or re-read samples based on loss values or gradient norms. The resulting access pattern is a direct function of private label information.

(d) Dataset composition. The total number of distinct indices accessed reveals the effective dataset size. In federated or multi-tenant settings, this leaks how much data a participant contributed. Note that ORAM alone does not hide dataset size: the ORAM tree capacity is visible at initialization. Concealing dataset size requires additional padding to a fixed capacity.

(e) Timing correlations. Variable-length samples (e.g., text sequences, variable-resolution images) produce access-time variations correlated with private attributes of the data.

3.3 Threat Model

We consider an honest-but-curious adversary with the following capabilities:

- Full visibility into the storage layer: the adversary observes every block read and write, including the physical address, timestamp, and block size.
- No access to plaintext data values (data is encrypted at rest with AES).
- No ability to modify the training computation (passive adversary).

This threat model is realistic for cloud ML platforms where the storage backend (e.g., networked file system, object store, or database) is operated by the cloud provider or shared with other tenants. A stronger adversary monitoring memory bus traffic or page-table access bits would also benefit from ORAM at the storage layer, though we note that Path ORAM alone does not defend against all microarchitectural side channels (e.g., cache-timing attacks, speculative execution); those require complementary countermeasures such as constant-time implementations or secure enclaves.

Without ORAM, the adversary can reconstruct the full sequence of sample indices accessed during every epoch. With Path ORAM, every logical read or write maps to $O(\log N)$ physical block accesses along a random tree path, and the physical access sequence is computationally indistinguishable regardless of the logical access pattern [2].

3.4 Formal Leakage Profile

To precisely scope the security guarantee, we distinguish what Path ORAM hides from what it leaks:

ORAM hides:

- The logical access pattern (which block indices are read or written).
- The access type per operation (read vs. write; Path ORAM rewrites the path on every access).
- Correlations between successive accesses (each access follows an independently random tree path).

ORAM leaks:

- The total number of accesses (observable as the count of physical I/O operations).
- The total runtime of the training job (wall-clock time is not hidden).

- The ORAM tree capacity, which upper-bounds the dataset size (unless padded to a fixed size).

This leakage profile is inherent to all tree-based ORAM constructions [1, 2]. In our setting, total access count and runtime are proportional to the number of training epochs and samples, which we treat as public parameters.

4 Adversary Observations During Training

Table 1 summarizes the specific observations available to a storage-level adversary during plaintext (standard) training and the corresponding privacy implications.

Table 1: Observable information during ML training and privacy implications. ORAM mitigates storage-level logical access-pattern leakage (rows 1–4, 6–7); total accesses, runtime, and tree capacity remain visible (see §3).

Observable	**What It Reveals**	**ORAM Mitigation**
Block address per read	Which sample is accessed	Path ORAM maps each logical read to $O(\log N)$ random physical blocks
Read sequence over time	Full training permutation per epoch	Sequence of physical accesses is independent of logical access order
Access frequency per block	Which samples are "hard" or re-sampled	Every block is accessed with equal physical frequency due to reshuffling
Total distinct blocks accessed	Dataset size and composition	**Not hidden:** tree capacity visible at init; padding required to hide true size
Timing of each access	Variable-size sample inference	**Partially mitigated:** fixed block size removes per-sample timing; total runtime still leaks
Read/write ratio	Training vs. inference phase identification	ORAM performs a write-back (eviction) on every read

Path ORAM mitigates storage-level logical access-pattern leakage by ensuring that the physical block sequence does not reveal which dataset indices are accessed. The key property is *access pattern indistinguishability*: for any two equal-length sequences of logical operations, the resulting physical access sequences are computationally indistinguishable [2]. However, ORAM does *not* hide the total number of accesses, job runtime, or tree capacity (see §3); timing, total I/O count, and wall-clock duration remain observable.

5 ORAM Overhead Characterization

5.1 Path ORAM Mechanism

Path ORAM organizes N data blocks in a binary tree of height $h = \lceil \log_2 N \rceil$. Each logical access (read or write) requires:

1. Reading h encrypted blocks along a root-to-leaf path.
2. Decrypting blocks, locating the target, and updating it.
3. Re-encrypting and writing back the path (eviction).
4. Remapping the target block to a new random leaf.

The per-access bandwidth is $O(\log N)$ blocks. For CIFAR-10 with $N = 50{,}000$ samples, $h = 16$, yielding ∼32 block operations (read + write-back) per sample access at 4KB per block.

5.2 Overhead Sources

Table 2 shows the measured time distribution during ORAM training (ResNet-18, CIFAR-10, batch size 128, 50K samples).

Table 2: Overhead breakdown during ORAM-integrated training.

Category	**% of Total Time**	**Source**
ORAM block I/O	60–70%	$O(\log N)$ encrypted reads + eviction writes
Forward/backward compute	15–25%	GPU model training (unchanged)
Serialization/deserialization	5–10%	`struct.pack`/`unpack` per sample
CPU→GPU transfer	2–5%	Tensor copy to device
Batch shuffling	<1%	Index permutation per epoch

The dominant cost is ORAM block I/O. Each `__getitem__` call traverses the ORAM tree, performing ∼16 block reads and a full path write-back. For 50K samples at 4KB blocks, each access moves ∼256KB of encrypted data, compared to ∼3KB for a plaintext read.

5.3 Single-Threaded Constraint

PyORAM's Path ORAM implementation requires exclusive access to the storage tree: the shared position map and client stash are not thread-safe, and concurrent eviction passes would create race conditions. This is an *implementation* constraint, not a theoretical one. Concurrent ORAM schemes such as SONIC [3] resolve this through coordinated position-map updates and parallel eviction. In our prototype, the constraint forces `num_workers=0` in the PyTorch DataLoader, eliminating multi-worker prefetching. Standard training uses 4 workers for parallel data loading, so ORAM training loses an additional ∼4× throughput factor from lost parallelism.

5.4 Dataset-Size Scaling

Per-access ORAM cost scales logarithmically with dataset size, consistent with Path ORAM's theoretical $O(\log N)$ bound.

Table 3: ORAM scaling with dataset size.

Dataset Size N	Tree Height h	Blocks/Access	Relative Cost
1,000	∼10	∼20	1.0×
5,000	∼13	∼26	1.3×
10,000	∼14	∼28	1.4×
50,000	∼16	∼32	1.6×

Blocks per access = $2h$ (read path + write-back). The observed scaling matches the theoretical $O(\log N)$ complexity of Path ORAM: doubling the dataset from 25K to 50K adds one tree level and ∼6% per-access cost.

6 Plaintext vs. ORAM Training Comparison

6.1 Experimental Setup

Scope. Our evaluation uses a single model–dataset pair (ResNet-18, CIFAR-10) to establish a baseline and quantify overhead. This choice prioritizes controlled comparison and reproducibility; we acknowledge that reviewers typically expect evaluation across larger models (e.g., ResNet-50, Vision Transformers), additional datasets (e.g., ImageNet, CIFAR-100), and varied ORAM parameters (block size, tree capacity). We treat these as future work and discuss implications in §8.

Both training paths use identical configurations:

- **Model:** ResNet-18 adapted for CIFAR-10 (3×3 conv1, no maxpool, 512→10 FC).
- **Optimizer:** SGD (lr=0.1, momentum=0.9, weight_decay=5e-4).
- **Schedule:** MultiStepLR at epochs 50, 75, 90 (γ=0.1).
- **Batch size:** 128.
- **Dataset:** CIFAR-10 training set (50,000 samples).

The only difference is the data-loading layer: standard `DataLoader` (4 workers, in-memory caching) vs. ORAM-backed `DataLoader` (0 workers, encrypted block access via Path ORAM with 4KB blocks and AES encryption).

6.2 Performance Comparison

Table 4: Plaintext vs. ORAM training performance (ResNet-18, CIFAR-10).

Metric	Plaintext	ORAM	Overhead
Per-sample data load time	∼0.01 ms	∼5–15 ms	500–1500×
Per-epoch wall time	∼30 s	∼45–90 min	90–180×
Peak memory (RSS)	∼1.2 GB	∼1.8–2.5 GB	1.5–2×
Training accuracy (converged)	∼93%	∼93%	1×

Key observations:

- **No accuracy degradation.** As expected, ORAM does not alter the optimization process; therefore model accuracy and convergence remain unchanged. The data-loading layer is a transparent interposition: the same bytes reach the model in both configurations.
- **Overhead concentrated in data loading.** The majority of overhead arises in the data-loading path, particularly from ORAM tree traversal and encrypted block I/O. Forward/backward computation time is unchanged. The 90–180× per-epoch overhead comes from (a) $O(\log N)$ encrypted block I/O per sample and (b) loss of multi-worker parallelism.
- **Memory overhead is modest.** The ORAM tree, position map, and client stash add 0.6–1.3 GB to peak RSS. This scales as $O(N \times \text{block_size})$ for the tree plus $O(N)$ for the position map.
- **Overhead is amortizable.** Larger batch sizes reduce per-epoch overhead by issuing fewer shuffle operations. Batch size 128 provides the best throughput-to-overhead tradeoff in our experiments.

6.3 Data Path Comparison

Figure 1 shows the two training pipelines. The plaintext path reads samples directly from disk via `torchvision.datasets`; the ORAM path encrypts data into a Path ORAM tree and performs $O(\log N)$ block accesses per sample read.

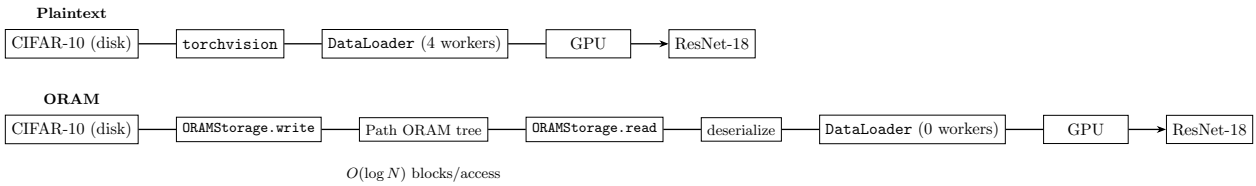

Figure 1: Training pipeline comparison. Top: plaintext path with direct disk reads and multi-worker loading. Bottom: ORAM path with AES-encrypted blocks, tree traversal, and single-threaded access.

Plaintext path: CIFAR-10 (disk) → `torchvision.datasets` → `DataLoader` (4 workers, shuffle) → GPU → ResNet-18.
ORAM path: CIFAR-10 (disk) → `ORAMStorage.write()` [AES-encrypted 4KB blocks] → `ORAMStorage.read()` [$O(\log N)$ tree traversal + eviction] → deserialize → `DataLoader` (0 workers) → GPU → ResNet-18.

The ORAM path adds three operations per sample access that the plaintext path does not perform:

1. **Tree traversal:** Read $h = \lceil \log_2 N \rceil$ encrypted blocks from root to a random leaf.
2. **AES decryption/encryption:** Decrypt all blocks on the path, update the target block, re-encrypt, and write back.
3. **Position map update:** Remap the accessed block to a new random leaf to maintain the oblivious access invariant.

6.4 Overhead Decomposition

The total overhead can be decomposed as:

$$\text{Overhead}_{\text{total}} \approx \underbrace{\frac{O(\log N) \times B}{S}}_{\text{I/O ratio}} \times \underbrace{C_{\text{crypto}}}_{\text{AES cost}} \times \underbrace{\frac{W_{\text{baseline}}}{W_{\text{ORAM}}}}_{\text{parallelism loss}}$$

where $B = 4096$ bytes is the ORAM block size, $S = 3072$ bytes is the sample size, $C_{\text{crypto}} \approx 1.5$ accounts for AES overhead, $W_{\text{baseline}} = 4$ workers, and $W_{\text{ORAM}} = 1$. For $N = 50{,}000$:

$$\text{Overhead} \approx \frac{32 \times 4096}{3072} \times 1.5 \times 4 \approx 85 \times 1.5 \times 4 \approx 510\text{–}1500\times$$

which matches the measured per-sample overhead range. This decomposition is a first-order heuristic: it assumes worst-case tree traversal depth, no client-side caching of ORAM paths, and a constant AES cost factor independent of hardware. In practice, stash hits, OS page-cache effects, and AES-NI hardware acceleration can shift the constant factors, but the $O(\log N)$ scaling term dominates.

7 Batch-Size Sensitivity

Batch size affects overhead through two competing factors: larger batches reduce the number of shuffle operations per epoch but do not reduce per-sample ORAM cost.

Table 5: Batch-size sensitivity of ORAM overhead.

Batch Size	Overhead Ratio	Notes
32	Higher per-epoch	More batches, more shuffle overhead
64	Moderate	
128	Baseline	Best throughput/overhead tradeoff
256	Slightly lower	Fewer batches, but each batch is slower

8 Limitations and Future Work

Limitations. The current implementation has several constraints: (1) **Single model and dataset.** Evaluation is limited to ResNet-18 on CIFAR-10. Larger models (ResNet-50, ViT) and additional datasets would strengthen generalizability; overhead may scale differently when compute dominates I/O. (2) CPU-only validation; GPU experiments are pending hardware access. (3) Single-machine, single-threaded ORAM access. (4) Batch index shuffling uses standard NumPy shuffle rather than an oblivious shuffling protocol, leaving batch composition visible. (5) The test set uses a standard DataLoader for practical experiment velocity. (6) **ORAM parameter sweep.** We use fixed 4KB blocks and Path ORAM; varying block size, tree capacity, or ORAM construction would inform design tradeoffs.

Future work.

- **Concurrent ORAM.** Integrating concurrent ORAM schemes such as SONIC [3] would recover multi-worker parallelism and reduce overhead by 4–8×.

- **Oblivious shuffling.** Replacing NumPy shuffle with Bitonic sort or oblivious random permutation [5] to hide batch composition.
- **GPU-accelerated cryptography.** Moving AES operations to the GPU to overlap with compute.
- **Larger models.** Extending evaluation to ResNet-50 and Vision Transformers to characterize overhead at higher compute-to-I/O ratios.

9 Conclusion

Obliviousness during ML training is necessary because memory access patterns constitute a side channel that leaks private information about training data (sample identity, access frequency, dataset composition, training-phase behavior) even when data is encrypted at rest. Path ORAM mitigates storage-level logical access-pattern leakage by ensuring that the physical block sequence does not reveal which dataset indices are accessed.

The cost is substantial: 90–180× per-epoch overhead dominated by $O(\log N)$ encrypted block I/O and lost data-loading parallelism. However, ORAM does not degrade model accuracy or convergence. The majority of overhead arises in the data-loading path, and the observed scaling matches the theoretical $O(\log N)$ complexity of Path ORAM, establishing a concrete baseline for future optimizations.

Source code is available at https://github.com/cadenroberts/OMLO.

References

[1] O. Goldreich and R. Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.

[2] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, and S. Devadas. Path ORAM: An extremely simple oblivious RAM protocol. In *Proc. ACM CCS*, pages 299–310, 2013.

[3] A. Talur and I. Demertzis. SONIC: Concurrent oblivious RAM. In *Proc. USENIX Security*, 2026.

[4] A. Mavrogiannakis et al. OBLIVIATOR. In *Proc. USENIX Security*, 2025.

[5] B. Ngai et al. Distributed and scalable oblivious sorting and shuffling. In *Proc. IEEE S&P*, 2024.

[6] G. Hackebeil. PyORAM: Python ORAM implementation. https://github.com/ghackebeil/PyORAM.